

STAT 4710: Homework 5

Ying Jin

Due: December 8, 2025 at 11:59am

Contents

Instructions	2
Case Study: Classifying Everyday Objects in CIFAR-10	4
1 Exploratory data analysis (18 points)	4
2 Logistic regression (30 points)	6
3 Multi-layer neural network (26 points)	7
4 Convolutional neural network (26 points)	7

```
options(scipen = 0, digits = 3) # controls number of significant digits printed
ggplot2::theme_set(ggplot2::theme_bw()) # set ggplot theme
options(warn=-1) # optional, to suppress warnings; comment out if you want to see warnings
options(show.signif.stars = FALSE)
# figure placement
knitr::opts_chunk$set(fig.pos = "!h", fig.align = "center")

library(torch)
library(torchvision)
library(luz)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(kableExtra)

##
## Attaching package: 'kableExtra'
##
## The following object is masked from 'package:dplyr':
##
##     group_rows

library(patchwork)
set.seed(4710); torch_manual_seed(4710)
```

Instructions

Materials and collaboration

The policy on allowed materials and collaboration is as stated on the Syllabus:

“Students are permitted to work together on homework assignments, but must write up and submit solutions individually. In particular, students may not copy each others’ solutions. Students may consult all course materials, textbooks, the internet, or AI tools (e.g. ChatGPT) to complete their homework. Students may not use solutions to problems that may be available online and/or from past iterations of the course. For each homework, students must disclose all classmates with whom they collaborated, which AI tools they used, and how they used them. Failure to do so will result in a 10-point penalty.”

In accordance with this policy,

Please disclose all classmates with whom you collaborated:

Please disclose which AI tools you used, and how you used them:

Failure to answer the above questions will result in a 10-point penalty.

Writeup

Use [homework-5-template.Rmd](#) as a starting point for your writeup, adding your solutions after `***Solution***`. Add your R code using code chunks and add your text answers using `***bold text***`. Consult the [preparing reports guide](#) for guidance on compilation, creation of figures and tables, and presentation quality. If the instructions ask you to “print a tibble”, you should print the tibble directly.

Programming

We encourage you to follow the `tidyverse` paradigm for data visualization, manipulation, and wrangling, as in the first homework, to practice tidyverse workflows (e.g., `filter`, `mutate`, `group_by`, `summarize`, `join`, `ggplot2`).

Students who strongly prefer to use Python for homework may do so. If you choose this option, you are **required** to present both code chunks and the corresponding outputs/interpretations in your PDF. In addition, it’s recommended that you:

- Use `pandas` for data manipulation (preferred, since it parallels `dplyr`).
- Use `plotnine` for visualization (preferred, since it parallels `ggplot2`). If using `seaborn`/`matplotlib`, you must still include the required plot elements (labels, 45° line, smoothing curve, facets, etc.).

Code completion. In some problems, the task is to complete an incomplete code chunk provided or simply to run a given code chunk. In those problems, in the template, there is `eval = FALSE` in the chunk header to prevent evaluation errors. You should remove `eval = FALSE` after you have completed the code to enable evaluation and printing of results.

Grading

The point value for each problem sub-part is indicated. There are 100 points possible on this homework, evaluated based on the correctness of the numerical results and plots. When a problem asks for a plot:

- **Axes and labels:** Always include informative axis labels and, if appropriate, titles.
- **Required elements:** If the instructions ask for a 45° line, smoothing curve, or facets, these must be included.
- **Clarity:** Points may be deducted if plots are cluttered, unreadable, or missing labels.

Submission

Compile your writeup to PDF and submit to Gradescope.

Case Study: Classifying Everyday Objects in CIFAR-10

In this homework, you'll prototype three increasingly powerful models (Logistic Regression → MLP → small CNN) on CIFAR-10, a classic benchmark of 10 everyday categories (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). Images are 32×32 RGB, making this a realistic “small-input/limited-compute” scenario.

You'll practice end-to-end model development: dataset handling, architecture design, training, and error analysis. Along the way, you'll connect model capacity to empirical performance and identify trade-offs that matter for deployment (speed, memory, confusion patterns). Keep notes—your conclusions should read like an internal engineering memo.

Let's first load the data.

```
tf_function <- function(img) {
  x <- torch_tensor(aperm(img, c(3,1,2)), dtype = torch_float()) / 255
  m <- torch_tensor(cifar10_mean)$view(c(-1,1,1))
  s <- torch_tensor(cifar10_std)$view(c(-1,1,1))
  return((x - m) / s)
}

cifar10_mean <- c(0.4914, 0.4822, 0.4465)
cifar10_std  <- c(0.2470, 0.2435, 0.2616)

train_ds_full <- cifar10_dataset("./data", train = TRUE, download = TRUE, transform = tf_function)
test_ds      <- cifar10_dataset("./data", train = FALSE, download = TRUE, transform = tf_function)

set.seed(4710)
n <- length(train_ds_full)
idx <- sample.int(n)
n_train <- floor(0.8 * n)
train_idx <- idx[1:n_train]
valid_idx <- idx[(n_train + 1):n]

batch_size <- 128
train_dl <- dataloader(dataset_subset(train_ds_full, train_idx), batch_size = batch_size)
valid_dl <- dataloader(dataset_subset(train_ds_full, valid_idx), batch_size = batch_size)
test_dl  <- dataloader(test_ds, batch_size = batch_size, shuffle = FALSE)

class_names <- c("airplane", "automobile", "bird", "cat", "deer",
                 "dog", "frog", "horse", "ship", "truck")
```

1 Exploratory data analysis (18 points)

Let's build some intuitions about the dataset by exploratory analysis.

1. (2 points) For the class labels in the data `train_ds_full`, print the proportion of each class. What do you find? [Hint: print out `train_ds_full` to see what it contains.]
2. (2 points) What is the dimension of an image? [Hint: print the size of some elements in `train_ds_full`.]
3. (3 points) Read the provided `tf_function()`. What does it do? Explain each line of the code. [Hint: You can print out some objects to get ideas if needed. For instance, for the third line

`torch_tensor(cifar10_mean)$view(c(-1,1,1))`, you can try print `cifar10_mean` as well as the output of this code.]

- (4 points) As preparation for plotting some images in the dataset, we are going to implement a helper function that turns a tensor (image) in the dataset, normalize it back to $[0,1]$ range, and converts it to a raster object for plotting. Complete the following incomplete function `tensor_to_raster()`.

```
tensor_to_raster <- function(x_chw, # transformed data in CHW format
                             mean = cifar10_mean,
                             std  = cifar10_std) {
  m <- # TO BE COMPLETED
  s <- # TO BE COMPLETED
  x_hwc <- (x_chw * s + m)$permute(c(2,3,1))
  a <- pmin(pmax(as.array(x_hwc), 0), 1)
  cols <- grDevices::rgb(a[,1], a[,2], a[,3]) # convert to colors
  dim(cols) <- dim(a)[1:2]
  return(grDevices::as.raster(cols))
}
```

- (5 points) We are now going to write a function that plots the images. As preparation, below is a code chunk in Lecture 1 programming exercise where we made a grid of plots for MNIST data. Please make the following changes to the code to make it work for CIFAR10 data:

- Rename the function as `plot_cifar_grid_ggplot()`.
- Modify the code so that it works for our CIFAR10 dataset `ds` passed as an argument to the function.
- Modify the definition of `img_array` so that we use all channels of the image.
- Modify the definition of `lab` to show the class name of the corresponding image.
- Modify the definition of `raster_img`, and use the `tensor_to_raster()` function you implemented above to convert the image tensor to a raster object.

```
plot_mnist_grid_ggplot <- function(ds, idxs = 1:9, ncol = 3) {
  plots <- lapply(idxs, function(i) {
    # get the i-th sample from the dataset
    ex <- ds[i]
    img_array <- as.array(ex[[1]][1,,]) # Extract 28x28 matrix
    lab <- as.integer(ex[[2]])
    raster_img <- as.raster(img_array)

    # Create ggplot
    ggplot() +
      annotation_raster(raster_img, xmin = 0, xmax = 1, ymin = 0, ymax = 1) +
      labs(title = paste("Label:", lab)) +
      theme_void() +
      theme(plot.title = element_text(hjust = 0.5, size = 10)) +
      coord_fixed()
  })

  # Arrange plots in grid
  wrap_plots(plots, ncol = ncol)
}
```

- (2 points) Use the above implemented function to visualize the first 6 images in `train_ds_full`.

2 Logistic regression (30 points)

We are now going to implement a logistic regression model (single-layer NN) for CIFAR-10 classification.

1. (2 points) In the first step, before feeding the images to the model, we need to flatten the image to vectors before inputting to the NN. In this case, what will be the dimension of the input to the NN?
2. (2 points) The output will be the logits for every class. What is the dimension of the output?
3. (10 points) Based on the answers to the questions, and use the in-class exercise, complete the following incomplete code chunk to define a logistic regression model class `LogisticRegression` that inherits from `nn_module`. The model should consist of a single linear layer that maps the input image (flattened to a vector) to the output logits for 10 classes.

```
LogisticRegression <- nn_module(  
  initialize = function() {  
    # TO BE COMPLETED  
  },  
  forward = function(x) {  
    # TO BE COMPLETED  
  }  
)
```

4. (8 points) Complete the following incomplete code chunk to instantiate the model, specify the loss function (cross-entropy loss for multi-class classification), the optimizer (RMSprop), and the metric (accuracy). We also set the learning rate `lr=1e-3` and the number of epochs `epochs=10` for training.

```
set.seed(4710) # for reproducibility  
  
fit_logistic <- LogisticRegression %>%  
  setup( # TO BE COMPLETED  
    ) %>%  
  set_opt_hparams(  
    # TO BE COMPLETED  
  ) %>%  
  fit(  
    # TO BE COMPLETED  
  )  
  
plot(fit_logistic)
```

5. (4 points) Based on the training curve, what do you observe about the training and validation accuracy? Do you think the model is underfitting or overfitting? Justify your answer.
6. (2 points) Make predictions on the test data using the trained model, and report the test accuracy.
7. (2 points) What is the total number of parameters in this model? Calculate it explicitly by hand. After this, use the following code to check if your calculation matches the output.

```
count_params <- function(module) sum(sapply(module$parameters, \(p) as.numeric(p$numel())))  
count_params(fit_logistic$model)
```

3 Multi-layer neural network (26 points)

Taking a step further, we are going to implement a multi-layer, fully-connected neural network.

1. (2 points) In the first step, we still flatten the image to vectors. And the output is still the logits for all classes. In this case, what will be the dimension of the input and output of the NN?
2. (10 points) Based on the answers to the questions, and use the in-class exercise, complete the following incomplete code chunk to define a multi-layer neural network class `MLP` that inherits from `nn_module`. The model should consist of two hidden layers with 256 and 128 units respectively, each followed by a ReLU activation function and dropout with rates 0.4 and 0.3 respectively; finally, we have an output layer that maps to the logits.

```
MLP <- nn_module(  
  initialize = function() {  
    # TO BE COMPLETED  
  },  
  forward = function(x) {  
    # TO BE COMPLETED  
  }  
)
```

3. (8 points) Complete the following incomplete code chunk to instantiate the model, specify the loss function (cross-entropy loss for multi-class classification), the optimizer (adam), and the metric (accuracy). We also set the learning rate `lr=1e-3` and the number of epochs `epochs=10` for training, with callbacks including early stopping (`patience=3`) and model checkpointing (saving the best model based on validation loss).

```
set.seed(4710) # for reproducibility  
fit_mlp <- MLP %>%  
  setup( # TO BE COMPLETED  
    ) %>%  
  set_opt_hparams(  
    # TO BE COMPLETED  
  ) %>%  
  fit(  
    # TO BE COMPLETED  
  )  
plot(fit_mlp)
```

4. (4 points) Based on the training curve, what do you observe about the training and validation accuracy? Do you think the model is underfitting or overfitting? Justify your answer.
5. (2 points) Make predictions on the test data using the trained model, and report the test accuracy. [Hint: when knitting the Rmd file, sometimes you'd get error `external pointer is not valid`. In this case, add `fit_mlp <- luz_load("PATH_TO_YOUR_SAVED_MODEL")` before evaluating on the test data.]

4 Convolutional neural network (26 points)

Finally, we are going to implement a small convolutional neural network (CNN). The structure we are using is as follows:

```

Input (3×32×32)
→ Conv(3→32, kernel_size=3) + ReLU
→ Conv(32→64, kernel_size=3) + ReLU
→ MaxPool(2×2)           # 28×28 → 14×14
→ Dropout(0.25)
→ Flatten (64×14×14=12544)
→ Linear(12544→256) + ReLU
→ Dropout(0.5)
→ Linear(256→10)         # logits

```

1. (2 points) In the first step, we feed the images in their original shape ($32 \times 32 \times 3$) to the model. And the output is still the logits for all classes. In this case, what will be the dimension of the input and output of the NN?
2. (10 points) Based on the answers to the questions, and use the in-class exercise, complete the following incomplete code chunk to define a small CNN class `SmallCNN` that inherits from `nn_module`. The model should consist of two convolutional layers followed by max-pooling and ReLU activation; then flatten the output and pass it through a fully-connected layer to get the logits.

```

SmallCNN <- nn_module(
  initialize = function() {
    # TO BE COMPLETED
  },
  forward = function(x) {
    # TO BE COMPLETED
  }
)

```

3. (8 points) Complete the following incomplete code chunk to instantiate the model, specify the loss function (cross-entropy loss for multi-class classification), the optimizer (adam), and the metric (accuracy). We also set the learning rate `lr=1e-3` and the number of epochs `epochs=10` for training, with callbacks including early stopping (`patience=3`) and model checkpointing (saving the best model based on validation loss).

```

set.seed(4710) # for reproducibility
fit_cnn <- SmallCNN %>%
  setup( # TO BE COMPLETED
    ) %>%
  set_opt_hparams(
    # TO BE COMPLETED
  ) %>%
  fit(
    # TO BE COMPLETED
  )
plot(fit_cnn)

```

4. (4 points) Based on the training curve, what do you observe about the training and validation accuracy? Do you think the model is underfitting or overfitting? Justify your answer.
5. (2 points) Make predictions on the test data using the trained model, and report the test accuracy.